

0.63.1. Trois formes ensemble — de base, ailleurs

Une spirale de plus, en bas à droite : le D autour de (14, 12).

1. Ce qu'il faut comprendre

C'est le **0.63**, avec une spirale **en bas à droite**, par le D.

C'est la **version de base** : la deuxième place, seule. Le 0.63.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <spirale>` → [0.58.3. La fonction spirale\(\) — tourner en spirale](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1347 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
spirale()      442 o ← main() – une fonction fournie par le langage
main()         34 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() – efface des cases
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 3    39 o sur place, dans spirale() + AdresseCase (10 o)
effacer() × 2   30 o sur place, dans spirale() + AdresseCase (10 o), EffacerCases (12 o)
image() × 3     9 o sur place, dans spirale(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
spirale() × 1   27 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

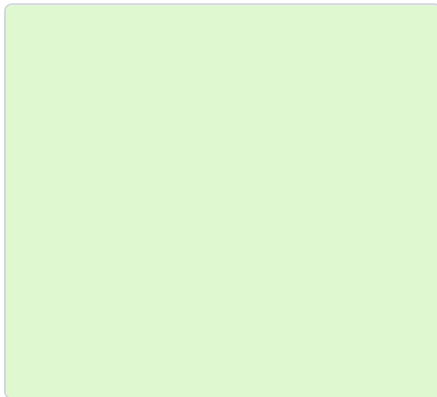
```
#include <spirale> // une tuile qui tourne en spirale
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    spirale(14, 12, ALPHABET[3], 2, 1, 100);           // le D, en bas à droite  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

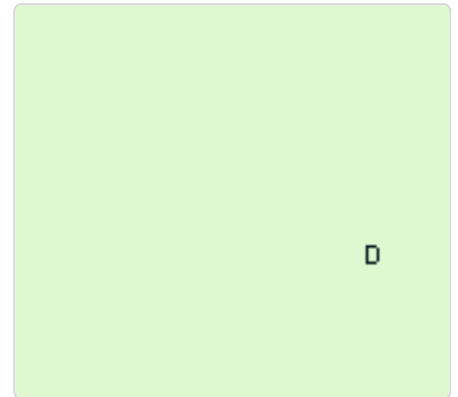
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Une spirale en bas à droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le D passe par (16, 10)

✓ le D finit en (14, 12) (14,12)

Ensuite : **0.63.2. Trois formes ensemble — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++